Лабораторная работа № 13. Программирование в командном процессоре ОС UNIX. Ветвления и циклы

НПИбд-03-25

Чжу Ино

2026-10-15

Содержание

1 Цель работы **2**

2 Задачи **2**

3 Теоретические сведения **2**
3.1 Команда getopts 2
3.2 Команда grep 3
3.3 Операторы управления 3
3.4 Прерывание цикла 3
3.5 Команды true и false 3
3.6 Условие if test -f 3
3.7 Отличие while от until 4

4 Выполнение лабораторной работы **4**
4.1 Скрипт 1: mygrep 4
4.2 Скрипт 2: C-программа 5
4.3 Скрипт 3: создание файлов 7
4.4 Скрипт 4: архивирование 9
4.5 Все скрипты 11

5 Выводы **12**

6 Контрольные вопросы **12**

7 Список литературы **12**

Список Иллюстраций

1 Скрипт mygrep 4
2 chmod 5
3 Запуск mygrep 5
4 Вывод в файл 5
5 C-программа 6
6 Компиляция 6
7 Запуск 6
8 Код возврата 7
9 Скрипт create 8
10 Запуск 8
11 create 3 9
12 Скрипт archive 9
13 Запуск archive 10
14 7 дней 10
15 Другой каталог 10
16 Все скрипты 11
17 Запуск всех 11

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задачи

1. Написать командный файл с использованием `getopts` и `grep`.
2. Написать программу на C, определяющую знак числа.
3. Написать командный файл для создания и удаления N файлов.
4. Написать командный файл для архивирования файлов.

3 Теоретические сведения

3.1 Команда getopts

Позволяет разбирать опции командной строки:

```
while getopts "i:o:p:Cn" opt; do
    case $opt in
```

```
        i) input=$OPTARG ;;
        o) output=$OPTARG ;;
        p) pattern=$OPTARG ;;
        C) case_sensitive="-i" ;;
        n) line_numbers="-n" ;;
    esac
done
```

3.2 Команда grep

```
grep [-опции] шаблон [файл...]
```

- `-i` — игнорировать регистр
- `-n` — номера строк

3.3 Операторы управления

- `if ... then ... fi`
- `while ... do ... done`
- `until ... do ... done`
- `for ... in ... do ... done`
- `case ... esac`

3.4 Прерывание цикла

- `break` — завершить цикл
- `continue` — завершить итерацию

3.5 Команды true и false

- `true` — всегда возвращает 0
- `false` — всегда возвращает 1

3.6 Условие if test -f

Проверяет, является ли файл обычным файлом.

3.7 Отличие while от until

- `while` — выполняется, пока условие истинно
- `until` — выполняется, пока условие ложно

4 Выполнение лабораторной работы

4.1 Скрипт 1: mygrep

```
#!/bin/bash
while getopts "i:o:p:Cn" opt; do
    case $opt in
        i) input=$OPTARG ;;
        o) output=$OPTARG ;;
        p) pattern=$OPTARG ;;
        C) case_sensitive="-i" ;;
        n) line_numbers="-n" ;;
    esac
done

if [ -n "$output" ]; then
    grep $case_sensitive $line_numbers "$pattern" "$input" > "$output"
else
    grep $case_sensitive $line_numbers "$pattern" "$input"
fi
```

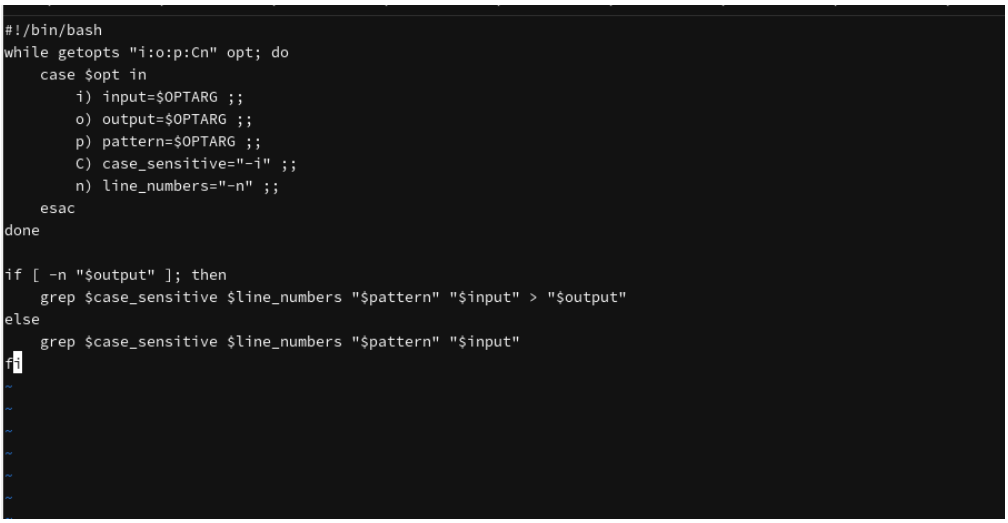


Рисунок 1: Скрипт mygrep

```
chmod +x mygrep.sh
```



Рисунок 2: chmod

```
echo -e "hello\nworld\nhello world" > test.txt
./mygrep.sh -i test.txt -p hello
```

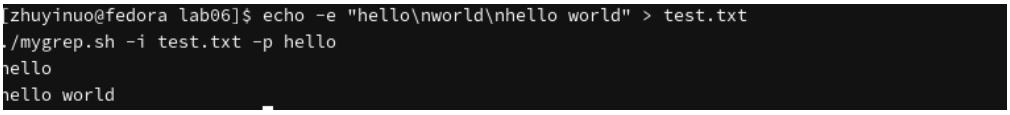


Рисунок 3: Запуск mygrep

```
./mygrep.sh -i test.txt -o result.txt -p hello
cat result.txt
```

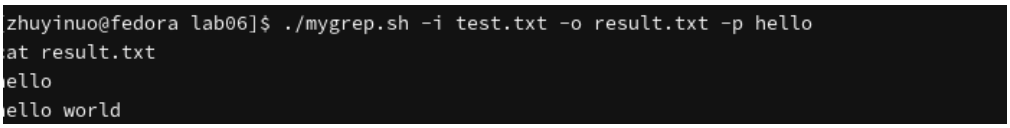


Рисунок 4: Вывод в файл

4.2 Скрипт 2: C-программа

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <number>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
    if (n > 0) {
        printf("Число %d больше нуля\n", n);
        return 0;
    } else if (n < 0) {
        printf("Число %d меньше нуля\n", n);
        return 2;
    } else {
```

```
        printf("Число равно нулю\n");
        return 3;
    }
}
```

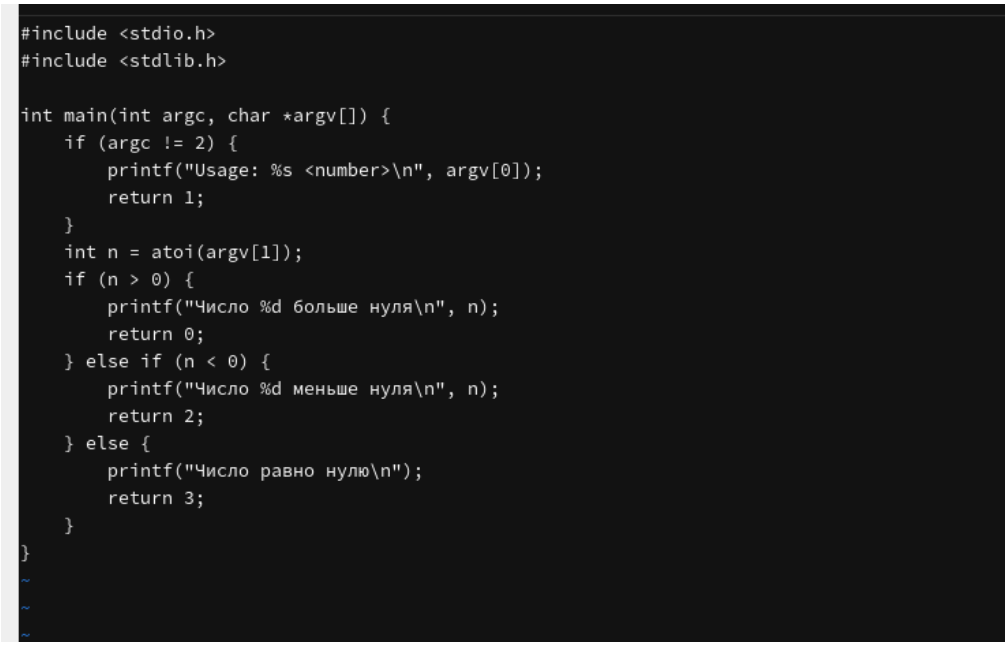


Рисунок 5: C-программа

```
gcc check_num.c -o check_num
ls -l check_num
```

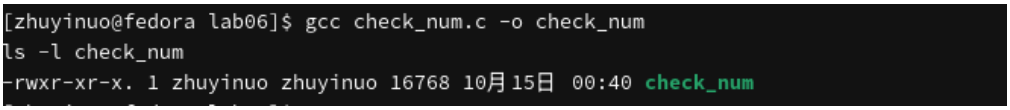


Рисунок 6: Компиляция

```
./check_num 5
./check_num -3
./check_num 0
```

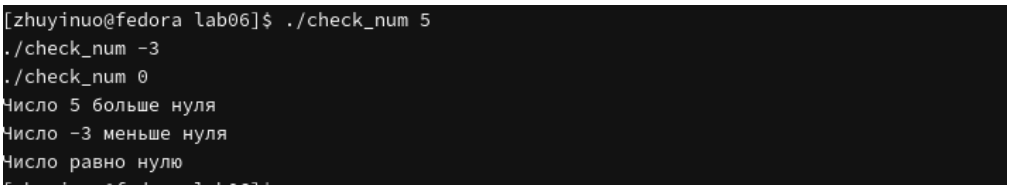


Рисунок 7: Запуск

```
./check_num 5
echo "Код возврата: $?"
./check_num -3
echo "Код возврата: $?"
./check_num 0
echo "Код возврата: $?"
```



Рисунок 8: Код возврата

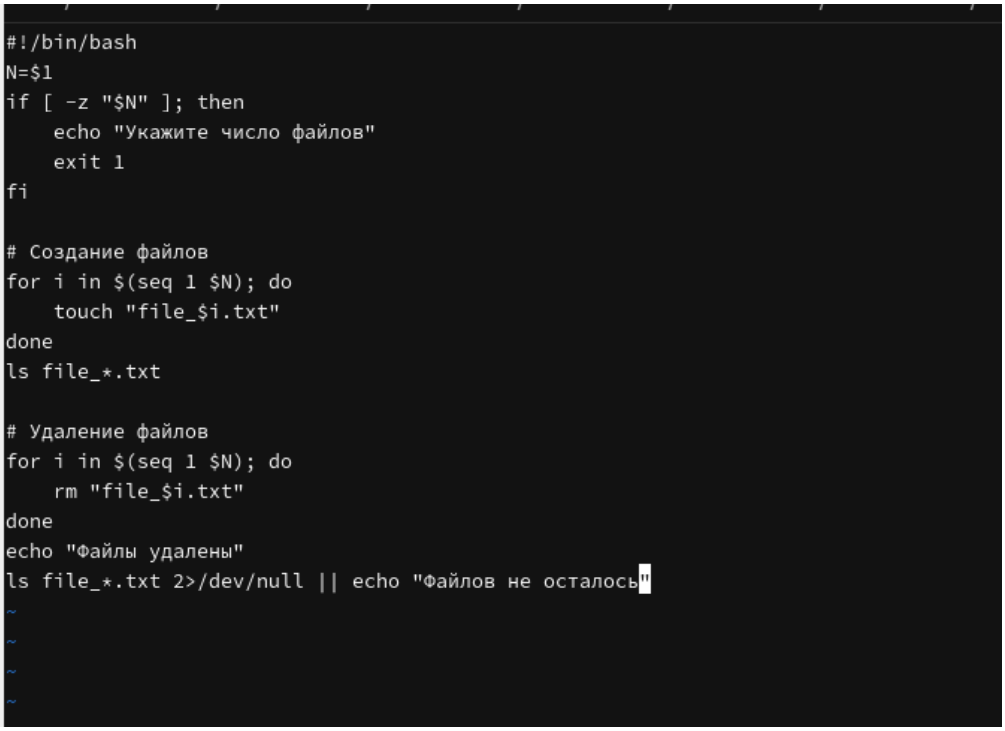
4.3 Скрипт 3: создание файлов

```
#!/bin/bash
N=$1
if [ -z "$N" ]; then
    echo "Укажите число файлов"
    exit 1
fi

for i in $(seq 1 $N); do
    touch "file_$i.txt"
done
ls file_*.txt

for i in $(seq 1 $N); do
    rm "file_$i.txt"
done
echo "Файлы удалены"
ls file_*.txt 2>/dev/null || echo "Файлов не осталось"
```

```
chmod +x create_files.sh
./create_files.sh 5
```

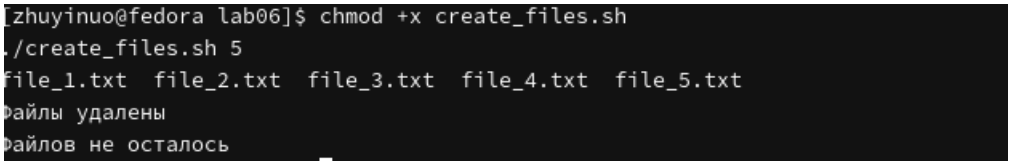


```
#!/bin/bash
N=$1
if [ -z "$N" ]; then
    echo "Укажите число файлов"
    exit 1
fi

# Создание файлов
for i in $(seq 1 $N); do
    touch "file_$i.txt"
done
ls file_*.txt

# Удаление файлов
for i in $(seq 1 $N); do
    rm "file_$i.txt"
done
echo "Файлы удалены"
ls file_*.txt 2>/dev/null || echo "Файлов не осталось"
~
~
~
~
```

Рисунок 9: Скрипт create



```
[zhuyinuo@fedora lab06]$ chmod +x create_files.sh
./create_files.sh 5
file_1.txt  file_2.txt  file_3.txt  file_4.txt  file_5.txt
Файлы удалены
Файлов не осталось
```

Рисунок 10: Запуск

```
./create_files.sh 3
```



Рисунок 11: create 3

4.4 Скрипт 4: архивирование

```
#!/bin/bash
DIR="${1:-.}"
OUT="${2:-archive.tar}"

tar -cvf "$OUT" "$DIR"

find "$DIR" -type f -mtime -7 -print0 | tar -cvf "recent_$OUT" --null -T -
```

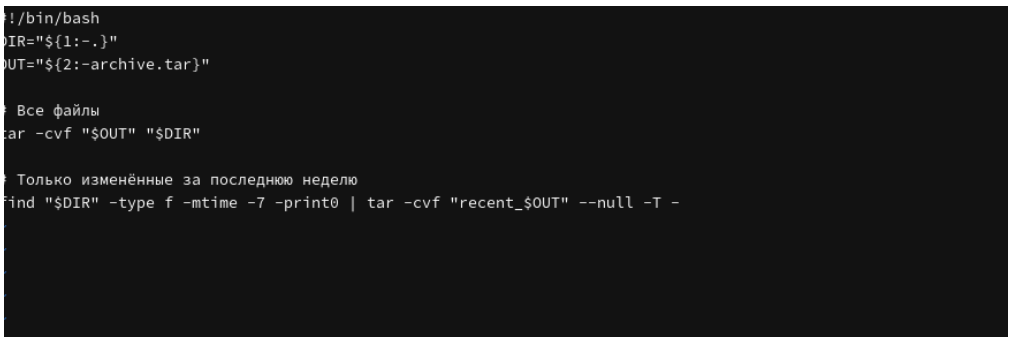


Рисунок 12: Скрипт archive

```
chmod +x archive.sh
./archive.sh ~/work/os/lab06 all_files.tar
ls -lh all_files.tar
```

```
ls -lh recent_all_files.tar
tar -tvf recent_all_files.tar
```

```
./archive.sh ~/work/os lab06.tar
ls -lh lab06.tar recent_lab06.tar
```

```
zhuyinuo@fedora lab06]$ vi archive.sh
zhuyinuo@fedora lab06]$ chmod +x archive.sh
/archive.sh ~/work/os/lab06 all_files.tar
s -lh all_files.tar
ar: 从成员名中删除开头的“/”
home/zhuyinuo/work/os/lab06/
home/zhuyinuo/work/os/lab06/hello.sh
home/zhuyinuo/work/os/lab06/backup.sh
home/zhuyinuo/work/os/lab06/args.sh
home/zhuyinuo/work/os/lab06/myls.sh
home/zhuyinuo/work/os/lab06/count.sh
home/zhuyinuo/work/os/lab06/mygrep.sh
home/zhuyinuo/work/os/lab06/test.txt
home/zhuyinuo/work/os/lab06/result.txt
home/zhuyinuo/work/os/lab06/check_num.c
home/zhuyinuo/work/os/lab06/check_num
home/zhuyinuo/work/os/lab06/create_files.sh
home/zhuyinuo/work/os/lab06/archive.sh
ar: /home/zhuyinuo/work/os/lab06/all_files.tar: archive cannot contain itself; not dumped
ar: 从成员名中删除开头的“/”
home/zhuyinuo/work/os/lab06/hello.sh
ar: 从硬连接目标中删除开头的“/”
home/zhuyinuo/work/os/lab06/backup.sh
home/zhuyinuo/work/os/lab06/args.sh
home/zhuyinuo/work/os/lab06/myls.sh
home/zhuyinuo/work/os/lab06/count.sh
home/zhuyinuo/work/os/lab06/mygrep.sh
home/zhuyinuo/work/os/lab06/test.txt
home/zhuyinuo/work/os/lab06/result.txt
home/zhuyinuo/work/os/lab06/check_num.c
home/zhuyinuo/work/os/lab06/check_num
home/zhuyinuo/work/os/lab06/create_files.sh
home/zhuyinuo/work/os/lab06/archive.sh
home/zhuyinuo/work/os/lab06/all_files.tar
ar: /home/zhuyinuo/work/os/lab06/recent_all_files.tar: archive cannot contain itself; not dumped
rw-r--r--. 1 zhuyinuo zhuyinuo 30K 10月15日  00:45 all_files.tar
```

Рисунок 13: Запуск archive

```
[zhuyinuo@fedora lab06]$ ls -lh recent_all_files.tar
 tar -tvf recent_all_files.tar
-rw-r--r--. 1 zhuyinuo zhuyinuo 60K 10月15日  00:45 recent_all_files.tar
-rwxr-xr-x zhuyinuo/zhuyinuo 204 2026-10-14 23:18 home/zhuyinuo/work/os/lab06/hello.sh
-rwxr-xr-x zhuyinuo/zhuyinuo 112 2026-10-15 00:19 home/zhuyinuo/work/os/lab06/backup.sh
-rwxr-xr-x zhuyinuo/zhuyinuo 157 2026-10-15 00:21 home/zhuyinuo/work/os/lab06/args.sh
-rwxr-xr-x zhuyinuo/zhuyinuo 256 2026-10-15 00:23 home/zhuyinuo/work/os/lab06/myls.sh
-rwxr-xr-x zhuyinuo/zhuyinuo 288 2026-10-15 00:25 home/zhuyinuo/work/os/lab06/count.sh
-rwxr-xr-x zhuyinuo/zhuyinuo 394 2026-10-15 00:38 home/zhuyinuo/work/os/lab06/mygrep.sh
-rw-r--r-- zhuyinuo/zhuyinuo  24 2026-10-15 00:39 home/zhuyinuo/work/os/lab06/test.txt
-rw-r--r-- zhuyinuo/zhuyinuo  18 2026-10-15 00:39 home/zhuyinuo/work/os/lab06/result.txt
-rw-r--r-- zhuyinuo/zhuyinuo 484 2026-10-15 00:40 home/zhuyinuo/work/os/lab06/check_num.c
-rwxr-xr-x zhuyinuo/zhuyinuo 16768 2026-10-15 00:40 home/zhuyinuo/work/os/lab06/check_num
-rwxr-xr-x zhuyinuo/zhuyinuo   391 2026-10-15 00:43 home/zhuyinuo/work/os/lab06/create_files.sh
-rwxr-xr-x zhuyinuo/zhuyinuo   243 2026-10-15 00:45 home/zhuyinuo/work/os/lab06/archive.sh
-rw-r--r-- zhuyinuo/zhuyinuo 30720 2026-10-15 00:45 home/zhuyinuo/work/os/lab06/all_files.tar
```

Рисунок 14: 7 дней

```
: 从硬连接目标中删除开头的“/”
ome/zhuyinuo/work/os/lab06/backup.sh
ome/zhuyinuo/work/os/lab06/args.sh
ome/zhuyinuo/work/os/lab06/myls.sh
ome/zhuyinuo/work/os/lab06/count.sh
ome/zhuyinuo/work/os/lab06/mygrep.sh
ome/zhuyinuo/work/os/lab06/test.txt
ome/zhuyinuo/work/os/lab06/result.txt
ome/zhuyinuo/work/os/lab06/check_num.c
ome/zhuyinuo/work/os/lab06/check_num
ome/zhuyinuo/work/os/lab06/create_files.sh
ome/zhuyinuo/work/os/lab06/archive.sh
ome/zhuyinuo/work/os/lab06/all_files.tar
ome/zhuyinuo/work/os/lab06/recent_all_files.tar
ome/zhuyinuo/work/os/lab06/lab06.tar
: /home/zhuyinuo/work/os/lab06/recent_lab06.tar: archive cannot contain itself; not dumped
```

Рисунок 15: Другой каталог

4.5 Все скрипты

```
ls -la ~/work/os/lab06/
```

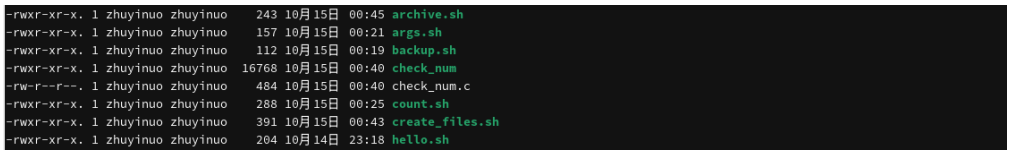


Рисунок 16: Все скрипты

```
./mygrep.sh -i test.txt -p hello && \
./check_num 5 && \
./create_files.sh 2 && \
./archive.sh . test.tar
```

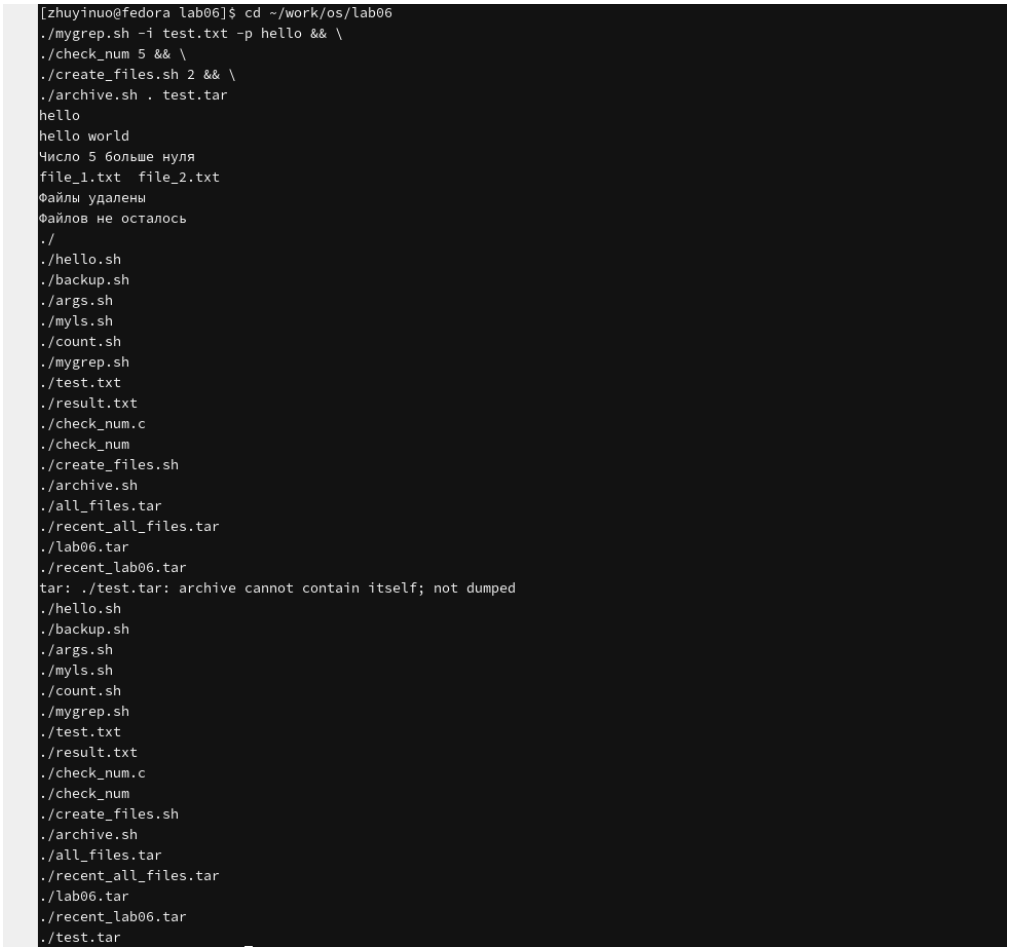


Рисунок 17: Запуск всех

5 Выводы

В ходе работы изучены логические управляющие конструкции и циклы в bash. Написаны четыре командных файла: `mygrep` с `getopts`, C-программа определения знака числа, скрипт создания/удаления файлов, скрипт архивирования с фильтром по времени.

6 Контрольные вопросы

1. Назначение команды getopts?

Разбор опций командной строки.

2. Отношение метасимволов к генерации имён файлов?

Метасимволы `*`, `?`, `[...]` используются для шаблонов имён файлов.

3. Операторы управления действиями.

`if`, `case`, `while`, `until`, `for`.

4. Операторы для прерывания цикла.

`break`, `continue`.

5. Для чего нужны команды false и true?

`true` всегда возвращает 0 (истина), `false` — 1 (ложь). Используются в условиях.

6. Что означает `if test -f man$s/$i.$s`?

Проверка, является ли файл `man$s/$i.$s` обычным файлом.

7. Различия while и until.

`while` — пока условие истинно, `until` — пока условие ложно.

7 Список литературы

1. Курячий К. В. Операционная система UNIX. — М. : ИНТУИТ, 2016.
2. Робачевский А., Немнюгин С., Стесик О. Операционная система UNIX. — БХВ-Петербург, 2010.
3. Колисниченко Д. Н. Самоучитель системного администратора Linux. — БХВ-Петербург, 2011.